

LABORATORY RECORD

Course:	Full Stack Web Development	Experiment No.:	3
Course Code:	23CM4121	Date:	30-08-2026
Student Name:	P.Dinesh Kumar	Roll No.:	A24126552101

1. TITLE

JavaScript Fundamentals: Variables, Control Structures, Arrays, Loops, and Functions

2. AIM

To write and execute a JavaScript program (week3.js) demonstrating core language fundamentals including variable declarations, conditional control structures (if-else), array operations, loops, functions, parameter passing, return values, and score average calculation.

3. OBJECTIVE

The objectives of this experiment are:

- To declare and log primitive variables (strings, numbers).
- To implement dynamic control flow using if-else condition checking.
- To inspect and mutate array data structures using methods such as .push().
- To iterate over array elements using a for loop.
- To define and invoke reusable functions with parameters and return values.
- To compute aggregate statistical metrics (sum, average) over numerical arrays.

4. THEORY

JavaScript is a lightweight, interpreted, object-oriented scripting language that powers dynamic behaviors on web platforms and server-side environments via Node.js.

Key Concepts:

Variables: let and const provide block-scoped variable declarations, replacing legacy function-scoped var.

Control Structures: if-else conditional statements evaluate boolean expressions to branch execution flow based on logical rules.

Arrays & Array Methods: Arrays store ordered collections of elements. Built-in methods like .push() append elements dynamically to the array's end.

Iterative Loops: for loops allow structured index-based iteration over array elements using zero-based indexing.

Functions & Aggregation: Functions modularize code by accepting arguments, returning calculated outputs, and performing iterative array computations such as sum and mean score evaluation.

Application Flow:

Variable Initialization → Conditional Logic → Array Mutation & Loop Iteration → Function Definition & Invocation → Average Calculation → Terminal Output

5. ALGORITHM / PROCEDURE

1. Initialize primitive variables name, age, and marks and output their values using console.log().
2. Evaluate marks >= 40 within an if-else block to determine and output pass/fail status.
3. Declare array subjects, access specific elements by index, append a new subject using subjects.push("Java"), and display the updated array.
4. Construct a standard for loop to iterate through all elements of subjects and display them line by line.
5. Define function greet(name) to print a personalized greeting message.
6. Define function add(a, b) to return the arithmetic sum of two numbers and print the returned result.
7. Create calculateAverage(marks) function that sums array elements via loop and returns the mean score.
8. Run the script using Node.js (node week3.js) and record the terminal output.

6. PROGRAM

week3.js

```
// 1. VARIABLES
let name = "Dinesh";
let age = 20;
let marks = 85;

console.log("Name:", name);
console.log("Age:", age);
console.log("Marks:", marks);

// 2. IF-ELSE STATEMENT
if (marks >= 40) {
    console.log("Result: Pass");
} else {
    console.log("Result: Fail");
}

// 3. ARRAY
let subjects = ["JavaScript", "Python", "HTML", "CSS"];

console.log("Subjects:", subjects);

// Accessing array elements
console.log("First subject:", subjects[0]);
console.log("Second subject:", subjects[1]);

// Adding an element
subjects.push("Java");

console.log("After adding Java:", subjects);

// 4. LOOP
console.log("All Subjects:");

for (let i = 0; i < subjects.length; i++) {
    console.log(subjects[i]);
}

// 5. FUNCTION
function greet(name) {
    console.log("Hello " + name + "!");
}

greet("Dinesh");
```

```
// 6. FUNCTION WITH PARAMETERS AND RETURN VALUE
function add(a, b) {
    return a + b;
}

let result = add(10, 20);

console.log("Sum:", result);

// 7. FUNCTION TO CALCULATE AVERAGE
function calculateAverage(marks) {
    let total = 0;

    for (let i = 0; i < marks.length; i++) {
        total = total + marks[i];
    }

    return total / marks.length;
}

let studentMarks = [80, 75, 90, 85];

let average = calculateAverage(studentMarks);

console.log("Average Marks:", average);
```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>let name = "Dinesh";</code>	Declares a block-scoped variable storing a string value.
<code>if (marks >= 40) { ... }</code>	Evaluates student mark against pass threshold to branch execution.
<code>subjects.push("Java");</code>	Appends a new string element dynamically to the end of the array.
<code>for (let i = 0; i < subjects.length; i++)</code>	Iterates through array elements using dynamic index positioning.
<code>function add(a, b) { return a + b; }</code>	Defines a parameter-driven function returning computed addition value.
<code>calculateAverage(marks)</code>	Accumulates array elements via loop to compute and return mean average.

8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	marks = 85 Check condition	Evaluate marks >= 40 Output Result: Pass	Result: Pass	Pass
TC-02	push("Java") & for loop execution	Append 'Java' to array & list all 5 subjects	Array updated with 5 items and displayed	Pass
TC-03	add(10, 20) & calculateAverage()	Sum: 30 & Average Marks: 82.5	Sum: 30 Average Marks: 82.5	Pass

9. OUTPUT

Output 1: JavaScript Fundamentals (node week3.js) Rendered Terminal

```
PS C:\Full Stack\Week 3> node week3.js
Name: Dinesh
Age: 20
Marks: 85
Result: Pass
Subjects: [ 'JavaScript', 'Python', 'HTML', 'CSS' ]
First subject: JavaScript
Second subject: Python
After adding Java: [ 'JavaScript', 'Python', 'HTML', 'CSS', 'Java' ]
All Subjects:
JavaScript
Python
HTML
CSS
Java
Hello Dinesh!
Sum: 30
Average Marks: 82.5
PS C:\Full Stack\Week 3>
```

10. RESULT

Thus, JavaScript fundamentals covering variables, control structures, arrays, loops, and functions were successfully implemented and verified across all test cases.